

SRMS - Student Result Management System

Team Documentation & Presentation Guide

Project Phase-2 Submission Team Size: 5 Members Project Completion: ~90%

Table of Contents

- 1. [Project Overview](#)
 - 2. [Tech Stack & Architecture](#)
 - 3. [Database Design & ERD](#)
 - 4. [Team Member Contributions](#)
 - 5. [Code File Locations](#)
 - 6. [API Routes & Endpoints](#)
 - 7. [System Workflow](#)
 - 8. [Presentation Q&A Preparation](#)
 - 9. [Seeded Test Data](#)
-

1. Project Overview

1.1 What is SRMS?

SRMS (Student Result Management System) is a comprehensive web application designed to manage student academic records in an educational institution. It provides separate interfaces for students, teachers, and coordinators to manage courses, enrollments, results, and generate transcripts.

1.2 Key Features

Feature	Description
Multi-Role Authentication	Separate login systems for Students, Teachers, and Coordinators
Department Management	CRUD operations for academic departments
Semester Management	Manage academic semesters
Course Management	Assign courses to departments and teachers
Student Enrollment	Enroll students in courses per semester
Result Entry	Teachers enter marks (mid, final, assignment, attendance)
GPA Calculation	Automatic grade point and GPA calculation
Result History	Track all modifications to student results
Transcript Generation	Download PDF transcripts per semester

1.3 User Roles

Role	Permissions
Student	View dashboard, search results, download transcripts
Teacher	Enter/edit results, view assigned courses, drop students
Coordinator	Full administrative access - manage departments, semesters, courses, enrollments, students, teachers

2. Tech Stack & Architecture

2.1 Technology Stack

Frontend:	Bootstrap 5 (HTML/CSS/JS)
Backend:	Laravel 11 (PHP 8.2+)
Database:	MySQL (XAMPP compatible)
PDF Generation:	Barryvdh/laravel-dompdf
Authentication:	Session-based (custom middleware)

2.2 Project Structure

srms/	
— app/	
— Http/	
— Controllers/	# Base controllers
— Middleware/	# Auth middleware (StudentAuth, TeacherAuth, CoordinatorAuth)
— Models/	# Eloquent models (11 models)
— Modules/	# Team member modules
— Sifat/	# Student Module
— Nazmul/	# Teacher & Coordinator Module
— Emon/	# Department & Semester Module
— Oishy/	# Enrollment & Results Module
— Mithila/	# Result History & Transcript Module
— database/	
— migrations/	# 14 migration files
— seeders/	# Database seeder with test data
— resources/	
— views/	# Blade templates (organized by module)
— routes/	
— web.php	# All application routes
— config/	# Laravel configuration

2.3 Design Pattern

The project follows **Laravel's MVC (Model-View-Controller)** architecture with module-based organization for team collaboration.

Request → Middleware (Auth Check) → Controller → Model (Database) → View
--

3. Database Design & ERD

3.1 Database Schema

Core Tables

1. departments

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
department_name	VARCHAR	UNIQUE, NOT NULL
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

2. semesters

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
semester_name	VARCHAR	UNIQUE, NOT NULL
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

3. students

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
student_id	VARCHAR	UNIQUE (e.g., S-24001)
full_name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
phone	VARCHAR(20)	UNIQUE
batch	VARCHAR	NOT NULL
enrollment_date	DATE	NOT NULL
password	VARCHAR	(hashed)
department_id	INT	FK → departments.id
semester_id	INT	FK → semesters.id
created_at	TIMESTAMP	

updated_at	TIMESTAMP	
------------	-----------	--

4. teachers

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
teacher_id	VARCHAR	UNIQUE (e.g., T-1001)
full_name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
designation	VARCHAR	NOT NULL
phone	VARCHAR(20)	UNIQUE
password	VARCHAR	(hashed)
role	VARCHAR	'teacher' or 'coordinator'
department_id	INT	FK → departments.id
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

5. courses

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
course_code	VARCHAR	UNIQUE (e.g., CSE101)
course_title	VARCHAR	NOT NULL
credit_hours	TINYINT	1-6
department_id	INT	FK → departments.id
teacher_id	INT	FK → teachers.id
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

6. enrollments

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
student_id	INT	FK → students.id
course_id	INT	FK → courses.id

semester_id	INT	FK → semesters.id
enrollment_date	DATE	NOT NULL
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
UNIQUE		(student_id, course_id, semester_id)

7. results

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
enrollment_id	INT	FK → enrollments.id, UNIQUE
mid_marks	TINYINT	0-30
final_marks	TINYINT	0-50
assignment_marks	TINYINT	0-10
attendance_marks	TINYINT	0-10
total_marks	TINYINT	0-100
letter_grade	VARCHAR(2)	A+, A, A-, B+, B, B-, C+, C, D, F
grade_point	DECIMAL(3,2)	0.00-4.00
gpa	DECIMAL(3,2)	0.00-4.00
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

8. result_histories

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
result_id	INT	FK → results.id
old_total_marks	TINYINT	Previous marks
old_grade	VARCHAR(2)	Previous grade
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

9. student_logins

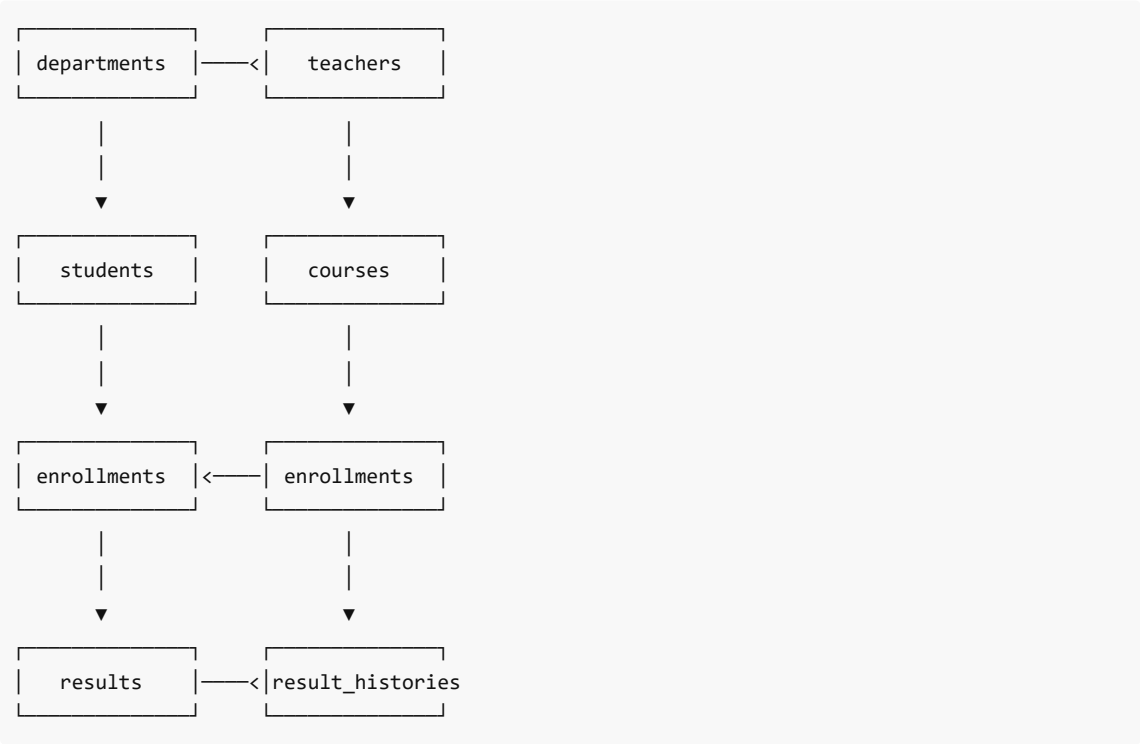
Column	Type	Constraints
--------	------	-------------

id	INT	PK, AUTO_INCREMENT
student_id	INT	FK → students.id
username	VARCHAR	UNIQUE (same as student_id)
password	VARCHAR	(hashed)
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

10. teacher_logins

Column	Type	Constraints
id	INT	PK, AUTO_INCREMENT
teacher_id	INT	FK → teachers.id
username	VARCHAR	UNIQUE (same as teacher_id)
password	VARCHAR	(hashed)
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

3.2 Entity Relationships



3.3 Normalization

The database follows **Third Normal Form (3NF)**:

- **1NF**: All tables have atomic values
 - **2NF**: No partial dependencies (single primary key)
 - **3NF**: No transitive dependencies (non-key attributes depend only on primary key)
-

4. Team Member Contributions

4.1 SIFAT (Team Leader) - Student Module

Role & Responsibilities

- **Team Leader**: Coordinated overall project development
- **Student Module Lead**: Complete student-facing functionality

Contributions

Backend (Controllers)

File	Purpose
app/Modules/Sifat/Controllers/StudentAuthController.php	Student login/logout with session management
app/Modules/Sifat/Controllers/StudentDashboardController.php	Dashboard with filters, GPA calculation, results display

Frontend (Views)

File	Purpose
resources/views/sifat/student-login.blade.php	Student login form
resources/views/sifat/dashboard.blade.php	Student dashboard with profile, results, filters, transcript download

Database

File	Purpose
database/migrations/2026_01_01_000003_create_students_table.php	Students table schema
database/migrations/2026_01_01_000009_create_student_logins_table.php	Student login credentials
app/Modules/Sifat/Database/students_tables.sql	SQL schema reference
app/Modules/Sifat/Queries/student_queries.sql	Sample SQL queries

Models

File	Purpose
app/Models/Student.php	Student model with relationships
app/Models/StudentLogin.php	Student login model

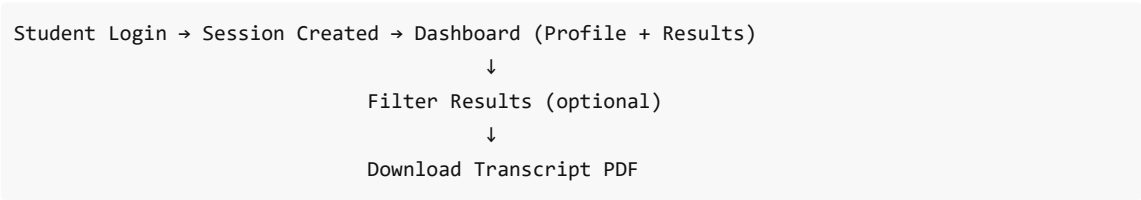
Middleware

File	Purpose
app/Http/Middleware/StudentAuth.php	Session-based student authentication

Key Features Implemented

- 1. **Authentication System:** Secure login with username/password validation
- 2. **Session Management:** Laravel session handling for logged-in students
- 3. **Dashboard Display:** Shows profile, department, semester, overall GPA
- 4. **Result Filtering:** Search by course code, semester, or grade
- 5. **GPA Calculation:** Automatic calculation based on credit hours and grade points
- 6. **Transcript Download:** Integration with Mithila's transcript module

Workflow



Code Location Summary

```
Backend: app/Modules/Sifat/Controllers/  
Frontend: resources/views/sifat/  
Database: database/migrations/000003*, 000009*  
Models: app/Models/Student.php, StudentLogin.php  
Middleware: app/Http/Middleware/StudentAuth.php  
Queries: app/Modules/Sifat/Queries/
```

4.2 NAZMUL - Teacher & Coordinator Module

Role & Responsibilities

- Teacher authentication and dashboard
- Coordinator administrative functions
- Student and teacher management

Contributions

Backend (Controllers)

File	Purpose
------	---------

app/Modules/Nazmul/Controllers/TeacherAuthController.php	Teacher/Coordinator login with role-based routing
app/Modules/Nazmul/Controllers/TeacherDashboardController.php	Coordinator and teacher dashboards
app/Modules/Nazmul/Controllers/CoordinatorTeacherController.php	Create new teachers (coordinator only)
app/Modules/Nazmul/Controllers/TeacherStudentController.php	Create new students (coordinator only)
app/Modules/Nazmul/Controllers/TeacherStudentDropController.php	Drop students from courses (teacher only)

Frontend (Views)

File	Purpose
resources/views/nazmul/teacher-login.blade.php	Teacher login form
resources/views/nazmul/coordinator-login.blade.php	Coordinator login form
resources/views/nazmul/dashboard.blade.php	Coordinator dashboard
resources/views/nazmul/teacher-dashboard.blade.php	Teacher dashboard
resources/views/nazmul/students/index.blade.php	Student list (for dropping)
resources/views/nazmul/students/create.blade.php	Student creation form
resources/views/nazmul/teachers/create.blade.php	Teacher creation form

Database

File	Purpose
database/migrations/2026_01_01_000004_create_teachers_table.php	Teachers table schema
database/migrations/2026_01_01_000010_create_teacher_logins_table.php	Teacher login credentials
app/Modules/Nazmul/Database/teachers_tables.sql	SQL schema reference
app/Modules/Nazmul/Queries/teacher_queries.sql	Sample SQL queries

Models

File	Purpose
app/Models/Teacher.php	Teacher model with department and course relationships
app/Models/TeacherLogin.php	Teacher login model

Middleware

File	Purpose
app/Http/Middleware/TeacherAuth.php	Session-based teacher authentication
app/Http/Middleware/CoordinatorAuth.php	Session-based coordinator authentication

Key Features Implemented

1. **Dual Authentication:** Separate login forms for teachers and coordinators
2. **Role-Based Access Control:** Different dashboards and permissions per role
3. **Teacher Management:** Create teachers with designations and departments
4. **Student Management:** Create students with batch and enrollment info
5. **Course Assignment Tracking:** Teachers see assigned courses and enrollment counts
6. **Student Dropping:** Teachers can remove students from their courses

Workflows

Coordinator Workflow

```
Coordinator Login → Dashboard
├─ Create Departments (via Emon's module)
├─ Create Semesters (via Emon's module)
├─ Create Courses (via Emon's module)
├─ Create Teachers
├─ Create Students
└─ Manage Enrollments (via Oishy's module)
```

Teacher Workflow

```
Teacher Login → Dashboard
├─ View Assigned Courses
├─ View Enrolled Students
├─ Enter/Edit Results (via Oishy's module)
├─ View Result Histories (via Mithila's module)
└─ Drop Students from Courses
```

Code Location Summary

```
Backend: app/Modules/Nazmul/Controllers/
Frontend: resources/views/nazmul/
Database: database/migrations/000004*, 000010*
Models: app/Models/Teacher.php, TeacherLogin.php
Middleware: app/Http/Middleware/TeacherAuth.php, CoordinatorAuth.php
Queries: app/Modules/Nazmul/Queries/
```

4.3 EMON - Department & Semester Module

Role & Responsibilities

- Department CRUD operations
- Semester management
- Course management with teacher assignment
- Database normalization

Contributions

Backend (Controllers)

File	Purpose
app/Modules/Emon/Controllers/DepartmentController.php	Full CRUD for departments
app/Modules/Emon/Controllers/SemesterController.php	Full CRUD for semesters
app/Modules/Emon/Controllers/CourseController.php	Full CRUD for courses with department/teacher assignment

Frontend (Views)

File	Purpose
resources/views/emon/departments/index.blade.php	Department list
resources/views/emon/departments/create.blade.php	Create department
resources/views/emon/departments/edit.blade.php	Edit department
resources/views/emon/semesters/index.blade.php	Semester list
resources/views/emon/semesters/create.blade.php	Create semester
resources/views/emon/semesters/edit.blade.php	Edit semester
resources/views/emon/courses/index.blade.php	Course list with teacher info
resources/views/emon/courses/create.blade.php	Create course (assign teacher)
resources/views/emon/courses/edit.blade.php	Edit course

Database

File	Purpose
database/migrations/2026_01_01_000001_create_departments_table.php	Departments schema
database/migrations/2026_01_01_000002_create_semesters_table.php	Semesters schema
database/migrations/2026_01_01_000005_create_courses_table.php	Courses schema with foreign keys
app/Modules/Emon/Database/department_semester_tables.sql	SQL schema reference
app/Modules/Emon/Queries/department_semester_queries.sql	Normalized SQL queries

Models

File	Purpose
app/Models/Department.php	Department model with student/teacher/course relationships
app/Models/Semester.php	Semester model with enrollment relationship
app/Models/Course.php	Course model with department, teacher, enrollment relationships

Key Features Implemented

- 1. **Department Management:** Create, read, update, delete departments
- 2. **Semester Management:** Create, read, update, delete academic semesters
- 3. **Course Management:** Assign courses to departments and teachers
- 4. **Foreign Key Relationships:** Proper database normalization
- 5. **Indexing:** Composite indexes for performance optimization

Database Design Decisions

- **Normalization:** All tables follow 3NF
- **Foreign Keys:** CASCADE on update, RESTRICT on delete
- **Unique Constraints:** Prevent duplicate entries
- **Indexes:** Added on frequently queried columns

Code Location Summary

```
Backend:  app/Modules/Emon/Controllers/
Frontend: resources/views/emon/
Database: database/migrations/000001*, 000002*, 000005*
Models:   app/Models/Department.php, Semester.php, Course.php
Queries:  app/Modules/Emon/Queries/
```

4.4 OISHY - Enrollment & Results Module

Role & Responsibilities

- Student course enrollment
- Result entry and management
- Grade calculation system
- GPA computation

Contributions

Backend (Controllers)

File	Purpose
app/Modules/Oishy/Controllers/EnrollmentController.php	Full CRUD for enrollments
app/Modules/Oishy/Controllers/ResultController.php	Full CRUD for results with history tracking

Backend (Services)

File	Purpose
app/Modules/Oishy/Services/GradeCalculator.php	Grade calculation logic (A+ to F)

Frontend (Views)

File	Purpose
resources/views/oishy/enrollments/index.blade.php	Enrollment list
resources/views/oishy/enrollments/create.blade.php	Create enrollment

resources/views/oishy/enrollments/edit.blade.php	Edit enrollment
resources/views/oishy/results/index.blade.php	Results list
resources/views/oishy/results/create.blade.php	Enter marks
resources/views/oishy/results/edit.blade.php	Edit marks (triggers history)

Database

File	Purpose
database/migrations/2026_01_01_000006_create_enrollments_table.php	Enrollments schema
database/migrations/2026_01_01_000007_create_results_table.php	Results schema
app/Modules/Oishy/Database/enrollments_results_tables.sql	SQL schema reference
app/Modules/Oishy/Queries/enrollment_result_queries.sql	Sample SQL queries

Key Features Implemented

Grade Calculation System

Total Marks = Mid (30) + Final (50) + Assignment (10) + Attendance (10) = 100

Grade Scale:

80-100 → A+ (4.00)
75-79 → A (3.75)
70-74 → A- (3.50)
65-69 → B+ (3.25)
60-64 → B (3.00)
55-59 → B- (2.75)
50-54 → C+ (2.50)
45-49 → C (2.25)
40-44 → D (2.00)
0-39 → F (0.00)

Enrollment Logic

- **Unique Constraint:** Student cannot enroll in same course twice in same semester
- **Foreign Keys:** Student, Course, Semester validation
- **Enrollment Tracking:** Date of enrollment recorded

Result Management

- **Access Control:** Teachers can only modify their own course results
- **History Tracking:** Automatic save to result_histories on edit
- **GPA Calculation:** Credit-hour weighted average

Code Location Summary

Backend: app/Modules/Oishy/Controllers/
Services: app/Modules/Oishy/Services/GradeCalculator.php
Frontend: resources/views/oishy/

Database: database/migrations/000006*, 000007*
Queries: app/Modules/Oishy/Queries/

4.5 MITHILA - Result History & Transcript Module

Role & Responsibilities

- Result modification history tracking
- PDF transcript generation
- Reporting and analytics queries

Contributions

Backend (Controllers)

File	Purpose
app/Modules/Mithila/Controllers/ResultHistoryController.php	View result modification history
app/Modules/Mithila/Controllers/TranscriptController.php	Generate and download transcript PDFs

Backend (Services)

File	Purpose
app/Modules/Mithila/Services/TranscriptPdfService.php	PDF generation using DomPDF

Frontend (Views)

File	Purpose
resources/views/mithila/histories/index.blade.php	Result history list
resources/views/mithila/transcript.blade.php	Transcript PDF template

Database

File	Purpose
database/migrations/2026_01_01_000008_create_result_histories_table.php	Result history schema
app/Modules/Mithila/Database/reporting_tables.sql	SQL schema reference
app/Modules/Mithila/Queries/reporting_queries.sql	Analytics and reporting queries

Models

File	Purpose
app/Models/ResultHistory.php	Result history model

Key Features Implemented

Result History Tracking

- **Automatic Capture:** When a teacher edits a result, the old values are saved

- **Audit Trail:** Tracks old_total_marks and old_grade
- **Timestamp:** Records when modifications occurred

Transcript Generation

- **PDF Download:** DomPDF library integration
- **Semester-Specific:** Generate transcript for specific semester
- **Includes:** Student info, course list, marks, grades, GPA
- **Credit Hours:** Shows total credits for semester

Code Location Summary

```
Backend:  app/Modules/Mithila/Controllers/  
Services: app/Modules/Mithila/Services/TranscriptPdfService.php  
Frontend: resources/views/mithila/  
Database: database/migrations/000008*  
Models:   app/Models/ResultHistory.php  
Queries:  app/Modules/Mithila/Queries/
```

5. Code File Locations

5.1 Complete File Structure by Member

SIFAT (Student Module)

```
Backend Controllers:  
  app/Modules/Sifat/Controllers/StudentAuthController.php  
  app/Modules/Sifat/Controllers/StudentDashboardController.php  
  
Frontend Views:  
  resources/views/sifat/student-login.blade.php  
  resources/views/sifat/dashboard.blade.php  
  
Database:  
  database/migrations/2026_01_01_000003_create_students_table.php  
  database/migrations/2026_01_01_000009_create_student_logins_table.php  
  app/Modules/Sifat/Database/students_tables.sql  
  app/Modules/Sifat/Queries/student_queries.sql  
  
Models:  
  app/Models/Student.php  
  app/Models/StudentLogin.php  
  
Middleware:  
  app/Http/Middleware/StudentAuth.php
```

NAZMUL (Teacher & Coordinator Module)

```
Backend Controllers:  
  app/Modules/Nazmul/Controllers/TeacherAuthController.php
```

```
app/Modules/Nazmul/Controllers/TeacherDashboardController.php
app/Modules/Nazmul/Controllers/CoordinatorTeacherController.php
app/Modules/Nazmul/Controllers/TeacherStudentController.php
app/Modules/Nazmul/Controllers/TeacherStudentDropController.php
```

Frontend Views:

```
resources/views/nazmul/teacher-login.blade.php
resources/views/nazmul/coordinator-login.blade.php
resources/views/nazmul/dashboard.blade.php
resources/views/nazmul/teacher-dashboard.blade.php
resources/views/nazmul/students/index.blade.php
resources/views/nazmul/students/create.blade.php
resources/views/nazmul/teachers/create.blade.php
```

Database:

```
database/migrations/2026_01_01_000004_create_teachers_table.php
database/migrations/2026_01_01_000010_create_teacher_logins_table.php
app/Modules/Nazmul/Database/teachers_tables.sql
app/Modules/Nazmul/Queries/teacher_queries.sql
```

Models:

```
app/Models/Teacher.php
app/Models/TeacherLogin.php
```

Middleware:

```
app/Http/Middleware/TeacherAuth.php
app/Http/Middleware/CoordinatorAuth.php
```

EMON (Department & Semester Module)

Backend Controllers:

```
app/Modules/Emon/Controllers/DepartmentController.php
app/Modules/Emon/Controllers/SemesterController.php
app/Modules/Emon/Controllers/CourseController.php
```

Frontend Views:

```
resources/views/emon/departments/index.blade.php
resources/views/emon/departments/create.blade.php
resources/views/emon/departments/edit.blade.php
resources/views/emon/semesters/index.blade.php
resources/views/emon/semesters/create.blade.php
resources/views/emon/semesters/edit.blade.php
resources/views/emon/courses/index.blade.php
resources/views/emon/courses/create.blade.php
resources/views/emon/courses/edit.blade.php
```

Database:

```
database/migrations/2026_01_01_000001_create_departments_table.php
database/migrations/2026_01_01_000002_create_semesters_table.php
database/migrations/2026_01_01_000005_create_courses_table.php
app/Modules/Emon/Database/department_semester_tables.sql
```

```
app/Modules/Emon/Queries/department_semester_queries.sql
```

Models:

```
app/Models/Department.php
app/Models/Semester.php
app/Models/Course.php
```

OISHY (Enrollment & Results Module)

Backend Controllers:

```
app/Modules/Oishy/Controllers/EnrollmentController.php
app/Modules/Oishy/Controllers/ResultController.php
```

Services:

```
app/Modules/Oishy/Services/GradeCalculator.php
```

Frontend Views:

```
resources/views/oishy/enrollments/index.blade.php
resources/views/oishy/enrollments/create.blade.php
resources/views/oishy/enrollments/edit.blade.php
resources/views/oishy/results/index.blade.php
resources/views/oishy/results/create.blade.php
resources/views/oishy/results/edit.blade.php
```

Database:

```
database/migrations/2026_01_01_000006_create_enrollments_table.php
database/migrations/2026_01_01_000007_create_results_table.php
app/Modules/Oishy/Database/enrollments_results_tables.sql
app/Modules/Oishy/Queries/enrollment_result_queries.sql
```

MITHILA (Result History & Transcript Module)

Backend Controllers:

```
app/Modules/Mithila/Controllers/ResultHistoryController.php
app/Modules/Mithila/Controllers/TranscriptController.php
```

Services:

```
app/Modules/Mithila/Services/TranscriptPdfService.php
```

Frontend Views:

```
resources/views/mithila/histories/index.blade.php
resources/views/mithila/transcript.blade.php
```

Database:

```
database/migrations/2026_01_01_000008_create_result_histories_table.php
app/Modules/Mithila/Database/reporting_tables.sql
app/Modules/Mithila/Queries/reporting_queries.sql
```

Models:

```
app/Models/ResultHistory.php
```

5.2 Shared/Core Files

```
app/Http/Controllers/Controller.php      # Base controller
app/Providers/AppServiceProvider.php     # Service providers
bootstrap/app.php                       # Application bootstrap
routes/web.php                          # All routes
config/app.php                          # App configuration
config/database.php                     # Database configuration
database/seeds/DatabaseSeeder.php       # Test data seeder
```

6. API Routes & Endpoints

6.1 Authentication Routes

Method	URI	Handler	Middleware
GET	/	Redirect to appropriate dashboard	-
GET	/student/login	Show student login form	-
POST	/student/login	Process student login	-
GET	/teacher/login	Show teacher login form	-
POST	/teacher/login	Process teacher login	-
GET	/coordinator/login	Show coordinator login form	-
POST	/coordinator/login	Process coordinator login	-

6.2 Student Routes

Method	URI	Handler	Middleware
GET	/student/dashboard	StudentDashboardController@index	student
GET	/student/transcript/{semesterId}	TranscriptController@download	student
POST	/student/logout	StudentAuthController@logout	student

6.3 Coordinator Routes

Method	URI	Handler	Middleware
GET	/coordinator/dashboard	TeacherDashboardController@index	coordinator
POST	/coordinator/logout	TeacherAuthController@logout	coordinator
GET	/coordinator/students/create	TeacherStudentController@create	coordinator
POST	/coordinator/students	TeacherStudentController@store	coordinator

GET	/coordinator/teachers/create	CoordinatorTeacherController@create	coordinator
POST	/coordinator/teachers	CoordinatorTeacherController@store	coordinator
CRUD	/departments	DepartmentController	coordinator
CRUD	/semesters	SemesterController	coordinator
CRUD	/courses	CourseController	coordinator
CRUD	/enrollments	EnrollmentController	coordinator

6.4 Teacher Routes

Method	URI	Handler	Middleware
GET	/teacher/dashboard	TeacherDashboardController@teacherIndex	teacher
POST	/teacher/logout	TeacherAuthController@logout	teacher
CRUD	/results	ResultController	teacher
GET	/result-histories	ResultHistoryController@index	teacher
GET	/teacher/enrollments	TeacherStudentDropController@index	teacher
DELETE	/teacher/enrollments/{id}	TeacherStudentDropController@destroy	teacher

7. System Workflow

7.1 Complete System Workflow

SYSTEM WORKFLOW

1. COORDINATOR SETUP PHASE

↳ Create Departments (CSE, EEE, BBA)

↳ Create Semesters (Spring 2025, Summer 2025, Fall 2025)

↳ Create Teachers (with designations)

↳ Create Courses (assign to department + teacher)

2. STUDENT ENROLLMENT PHASE

↳ Coordinator creates student accounts

↳ Coordinator enrolls students in courses per semester

3. RESULT ENTRY PHASE

- |
- |→ Teacher logs in
- |
- |→ View assigned courses and enrolled students
- |
- |→ Enter marks (mid + final + assignment + attendance)
- |
- |→ System calculates grade and GPA automatically

4. RESULT MODIFICATION PHASE (if needed)

- |
- |→ Teacher edits marks
- |
- |→ System saves old result to result_histories
- |
- |→ System updates with new marks

5. STUDENT VIEWING PHASE

- |
- |→ Student logs in
- |
- |→ View dashboard with all results
- |
- |→ Filter by course/semester/grade
- |
- |→ Download transcript PDF

7.2 Authentication Flow

AUTHENTICATION FLOW

STUDENT LOGIN:

Login Form → Validate Credentials → Check student_logins table
↓
Success: Store session (student_id, student_name) → Redirect to dashboard
↓
Failure: Return with error message

TEACHER LOGIN:

Login Form → Validate Credentials → Check teacher_logins table
↓
Success: Check role (teacher/coordinator) → Store session → Redirect accordingly
↓
Failure: Return with error message

MIDDLEWARE CHECKS:

student middleware: Check session('student_id')

```
teacher middleware: Check session('teacher_id') && session('teacher_role') === 'teacher'
coordinator middleware: Check session('teacher_id') && session('teacher_role') ===
'coordinator'
```

7.3 Grade Calculation Flow

INPUT: Mid (0-30) + Final (0-50) + Assignment (0-10) + Attendance (0-10)

|

▼

CALCULATE: Total = Mid + Final + Assignment + Attendance

|

▼

GRADE DETERMINATION:

	80-100	→ A+ (4.00)	
	75-79	→ A (3.75)	
	70-74	→ A- (3.50)	
	65-69	→ B+ (3.25)	
	60-64	→ B (3.00)	
	55-59	→ B- (2.75)	
	50-54	→ C+ (2.50)	
	45-49	→ C (2.25)	
	40-44	→ D (2.00)	
	0-39	→ F (0.00)	

|

▼

OUTPUT: Letter Grade + Grade Point + GPA

8. Presentation Q&A Preparation

8.1 Technical Questions

Q: What is the architecture of your application?

A: Our application follows Laravel's MVC (Model-View-Controller) architecture with a module-based organization. We have separate modules for each team member's contributions, with shared models and middleware.

Q: How did you handle authentication without Laravel Breeze/Fortify?

A: We implemented custom session-based authentication using Laravel middleware. The `StudentAuth`, `TeacherAuth`, and `CoordinatorAuth` middleware check session variables and redirect unauthenticated users to their respective login pages.

Q: Explain your database normalization approach.

A: Our database follows Third Normal Form (3NF):

- 1NF: All values are atomic
- 2NF: No partial dependencies (single primary keys)

- 3NF: Non-key attributes depend only on primary keys

We used foreign keys with CASCADE ON UPDATE and RESTRICT ON DELETE to maintain referential integrity.

Q: How is GPA calculated in your system?

A: $GPA = \frac{\sum(\text{Grade Point} \times \text{Credit Hours})}{\sum(\text{Credit Hours})}$

For each course, the grade point (0.00-4.00) is multiplied by credit hours, summed across all courses, then divided by total credit hours.

Q: How do you track result modifications?

A: When a teacher edits a result, our `ResultController@update` method automatically creates a new `ResultHistory` record with the old `total_marks` and `old_grade` before updating the result.

8.2 Module-Specific Questions

SIFAT - Student Module

Q: How does the student filter system work? A: The `StudentDashboardController` uses Laravel query scopes to filter enrollments by `course_code` (LIKE search), `semester_id` (exact match), and `letter_grade` (exact match). Multiple filters can be combined.

Q: How does transcript download work? A: The student requests a transcript for a specific semester ID. The `TranscriptController` fetches all enrollments with results for that student and semester, calculates GPA, and passes data to DomPDF for PDF generation.

NAZMUL - Teacher & Coordinator Module

Q: What's the difference between teacher and coordinator? A: Both use the `teachers` table but have different `role` values. Coordinators have administrative access to manage departments, semesters, courses, students, and teachers. Teachers can only manage results for their assigned courses.

Q: How did you implement role-based access control? A: We created separate middleware (`TeacherAuth` and `CoordinatorAuth`) that check both the session's `teacher_id` and `teacher_role`. Routes are protected by appropriate middleware groups.

EMON - Department & Semester Module

Q: Why is course assigned to both department and teacher? A: The course belongs to a department (for curriculum organization) and is taught by a teacher (for result entry permissions). This allows coordinators to see which department offers which courses and which teacher teaches it.

Q: How do you handle deletion of departments with students? A: We used RESTRICT ON DELETE for foreign keys. If a department has students, the deletion is blocked to maintain data integrity.

OISHY - Enrollment & Results Module

Q: How do you prevent duplicate enrollments? A: The `enrollments` table has a composite unique constraint on (`student_id`, `course_id`, `semester_id`), preventing the same student from enrolling in the same course twice in one semester.

Q: How is the grade calculation enforced? A: The `GradeCalculator` service handles all grade calculations in one place. Both `ResultController@store` (creating results) and `ResultController@update` (editing) use this service

to ensure consistent grading.

MITHILA - Result History & Transcript Module

Q: Why do you need result history? **A:** Result history serves as an audit trail. If a student's grade is modified, we have a record of what the previous grade was, who changed it (via session), and when.

Q: How is the PDF transcript generated? **A:** We use the Barryvdh/laravel-dompdf package. The `TranscriptPdfService` loads a Blade view with student and result data, then converts it to PDF.

8.3 Project Management Questions

Q: How did you organize work among 5 team members?

A: Each team member was assigned a specific module based on the database schema. We created a module folder structure under `app/Modules/` to keep our code organized and prevent merge conflicts.

Q: What challenges did you face and how did you overcome them?

A: One challenge was ensuring database consistency. We solved this by defining foreign key relationships and using `RESTRICT ON DELETE` to prevent orphaned records. Another challenge was coordinating authentication across modules, which we solved by creating shared middleware in the `app/Http/Middleware/` folder.

Q: What would you improve if you had more time?

A: We would add:

- Email notifications for grade publication
- Grade appeal system
- Performance analytics dashboard
- RESTful API for mobile app integration

9. Seeded Test Data

9.1 Login Credentials

Role	Username	Password	Name
Student	S-24001	password	Sifat Mazib
Student	S-24002	password	Tanvir Ahmed
Student	S-24003	password	Samiha Rahman
Student	S-24004	password	Nusrat Jahan
Teacher	T-1001	password	Md. Rahman (Professor)
Teacher	T-1002	password	Mahmudul Hasan (Lecturer)
Coordinator	C-9001	password	Coordinator Rahman

9.2 Departments

- CSE (Computer Science & Engineering)
- EEE (Electrical & Electronic Engineering)
- BBA (Bachelor of Business Administration)

9.3 Semesters

- Spring 2025
- Summer 2025
- Fall 2025

9.4 Courses

Code	Title	Credits	Department	Teacher
CSE101	Structured Programming	3	CSE	Md. Rahman
CSE102	Data Structures	3	CSE	Mahmudul Hasan
CSE201	Database Systems	3	CSE	Md. Rahman
EEE101	Basic Electronics	3	EEE	Farhana Yasmin
BBA101	Principles of Management	3	BBA	Saiful Islam

9.5 Sample Results

Student	Course	Mid	Final	Assign	Attend	Total	Grade
Sifat Mazib	CSE101	25	42	8	9	84	A+
Sifat Mazib	CSE102	22	38	7	9	76	A
Tanvir Ahmed	CSE201	20	35	8	8	71	A-
Samiha Rahman	EEE101	18	30	7	7	62	B
Nusrat Jahan	BBA101	24	40	8	9	81	A+

Appendix: Quick Reference

Important Commands

```
# Setup
composer install
php artisan key:generate
php artisan migrate --seed

# Development
php artisan serve
```

```
# Database
php artisan migrate          # Run migrations
php artisan db:seed         # Seed data
php artisan migrate:fresh --seed # Fresh start

# If PHP not recognized (Windows/XAMPP)
set PATH=C:\xampp\php;%PATH%
```

Key File Paths

- Project Root: C:\Users\sifat\OneDrive\Desktop\SRMS\srms
- Routes: routes/web.php
- Middleware: app/Http/Middleware/
- Models: app/Models/
- Views: resources/views/
- Migrations: database/migrations/
- Config: config/database.php

ERD Location

- Image: ERD_SRMS.png
- Source: ERD_SRMS (Graphviz format)
- Workflow: ERD_Workflow.png

Document Version: Phase-2 Final **Last Updated:** May 2026 **Team:** SRMS Development Team